

Project Design Phase-II Technology Stack (Architecture & Stack)

Date	17 July 2026
Team ID	
Project Name	SB Stocks – Stock Trading Application (MERN Stack)
Maximum Marks	4 Marks

Technical Architecture:

SB Stocks is built on the MERN stack (MongoDB, Express.js, React.js, Node.js). The React client talks to an Express REST API over HTTPS for authentication, trading and wallet operations, while a lightweight ticker engine pushes live price updates to the dashboard. All application data is persisted in MongoDB.

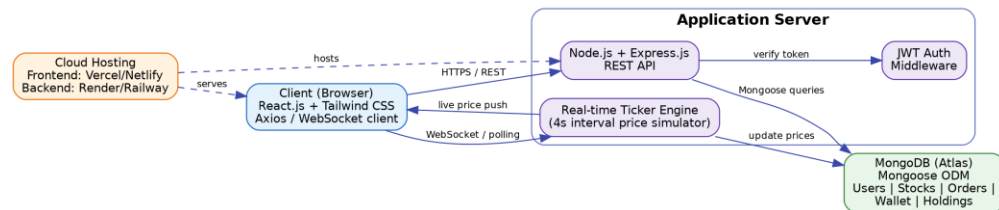


Figure 1: Technical Architecture – SB Stocks (MERN Stack)

Table-1 : Components & Technologies:

S.No	Component	Description	Technology
1.	User Interface	How the trader interacts with the application — login/register, dashboard, stock detail, portfolio, history and profile screens.	React.js, Tailwind CSS, React Router
2.	Application Logic-1	REST API for authentication, user, wallet and trade endpoints.	Node.js, Express.js
3.	Application Logic-2	Real-time ticker engine that simulates and pushes live stock price updates.	Node.js interval/WebSocket-based ticker engine
4.	Application Logic-3	Verifies the JWT on protected routes and attaches the trading user to each request.	JWT (jsonwebtoken), bcrypt.js
5.	Database	Stores users, stocks, orders, wallet ledger and holdings.	MongoDB, Mongoose ODM
6.	Cloud Database	Managed database service used in deployment.	MongoDB Atlas
7.	File Storage	No user file uploads are required by the current feature set.	Not applicable
8.	External API-1	Not used — market prices are generated/simulated internally by the ticker engine rather than pulled from a paid market-data provider.	Not applicable
9.	External API-2	Reserved for a future integration such as an email/OTP provider for registration confirmation.	Nodemailer / SMTP (planned)
10.	Machine Learning Model	Not part of the current scope.	Not applicable
11.	Infrastructure (Server / Cloud)	Frontend and backend are deployed separately as static and Node hosting respectively. Local Server Configuration: Node.js runtime with local MongoDB instance. Cloud Server Configuration: Frontend on Vercel/Netlify; backend on Render/Railway; database on MongoDB Atlas.	Vercel/Netlify, Render/Railway, MongoDB Atlas

The Dashboard below shows the Real-time Data ticker (4-second refresh engine) and MongoDB-backed schema referenced in Table-1.

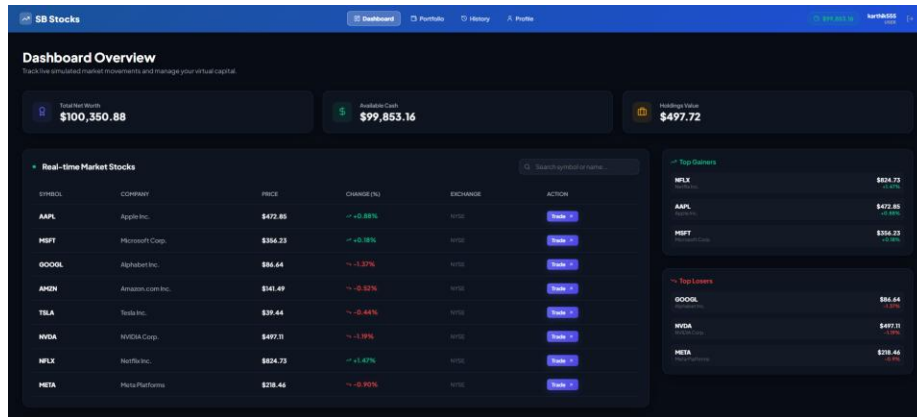


Figure 2: Dashboard Overview showing the real-time ticker engine, live market stocks table, and MongoDB-synced portfolio widgets

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1.	Open-Source Frameworks	Frontend and backend are both built entirely on open-source frameworks and libraries.	React.js, Node.js, Express.js, Mongoose
2.	Security Implementations	Passwords are hashed before storage; protected routes require a valid bearer token; role field restricts access to admin-only actions.	bcrypt password hashing, JWT authentication, role-based access control (User/Admin)
3.	Scalable Architecture	3-tier architecture (React client, Express API, MongoDB) keeps the UI, business logic and data layers independent so any tier can scale on its own.	MERN 3-tier architecture
4.	Availability	Frontend and backend are deployed on separate managed cloud platforms with automatic restarts; MongoDB Atlas provides replica sets for the database tier.	Vercel/Netlify, Render/Railway, MongoDB Atlas replica set
5.	Performance	The ticker engine refreshes prices on a 4-second interval; MongoDB indexes on user and symbol fields keep portfolio, dashboard and history queries responsive.	Interval-based ticker engine, MongoDB indexing

References:

<https://www.mongodb.com/mern-stack>

<https://reactjs.org/>

<https://expressjs.com/>

<https://c4model.com/>